13. GUIDE QUESTIONS

Answer the following questions in your final report:

1. **Data Structure Selection & Trade-Offs:** Why are dynamic arrays preferred over fixed-size static arrays for unpredictable batch workloads, and what hidden computational costs occur during memory reallocations?

   ANSWER: Static arrays are annoying for unpredictable workloads because if you make them too small, your app crashes, and if you make them too big, you waste memory. Dynamic arrays solve this by growing on demand, but that growth isn't free. Every time the array hits its limit, it has to allocate a whole new chunk of memory (usually double the size), copy every single item over one by one ($O(N)$ operation), and free the old block. It works fine most of the time, but those resize moments cause sudden performance lag.

2. **Memory Hygiene & Lifecycle:** Explain the role of the C++ destructor (~DynamicArray()) in managing heap allocation safety. What system failure occurs if delete[] is omitted in long-running C++ applications?

   ANSWER: In C++, heap memory doesn't clean itself up—you have to do it manually. The destructor (~DynamicArray()) automates this using RAII, meaning as soon as the array object goes out of scope, delete[] data runs automatically. If you forget delete[] in a long-running app, you get a memory leak. The program keeps grabbing RAM without returning it, eventually slowing down your machine or forcing the OS to crash the program with a memory error.

3. **Defensive Programming & Bounds Enforcement:** Contrast the behavior of raw array indexing (array[i]) with bounds-checked operations. Why is missing upper-bound validation (idx >= size) dangerous in security-critical codebases?

   ANSWER: Raw indexing like array[i] just jumps straight to a memory address without checking if it's safe—if i is invalid, it reads or overwrites random memory, causing unexpected behavior. Bounds checking prevents this by verifying $0 \leq idx < size$ before doing anything. Leaving out upper-bound checks (idx >= size) is dangerous in real-world software because attackers can exploit buffer overflows to corrupt adjacent data, read sensitive info, or crash the program.

4. **Algorithmic Overhead & Code Smells:** How did eliminating redundant nested loops inside the array deletion routine change the runtime performance of the system?

   ANSWER: The original deletion code had a pointless nested loop that made shifting elements messy and inefficient. Even though it technically didn't run as bad as $O(N^2)$ due to a break statement, the structure was confusing and threw off CPU execution. Cleaning it up into a single-pass loop ($O(N)$) stripped away all that unnecessary code, reduced CPU branching overhead, and made the logic way easier to read.

5. **Value of Peer Code Audits:** Why are structured peer code reviews essential in commercial software development prior to merging feature branches into production repositories?

ANSWER: Peer reviews are important because it's hard to spot your own mistakes after staring at code for hours. Having a teammate review your work helps catch logic bugs, memory leaks, and bad formatting before pushing code to production. It keeps the whole team's code consistent and prevents broken updates from hitting real users.

14. REFLECTION QUESTIONS

Include thoughtful responses to the following reflection prompts in your submission:

1. *Experiential Reflection:* What was the most challenging defect to locate inside the legacy codebase, and how did systematic unit testing assist you in pinpointing the root cause compared to random debugging?

   ANSWER: The most challenging defects to locate in the legacy codebase were the off-by-one traversal error in `printAll()` and the silent heap leak in `add_item()`, as informal debugging outputting garbage memory obscured the underlying boundary issue. Systematic unit testing paired with runtime sanitizers like AddressSanitizer (`-fsanitize=address`) efficiently isolated the exact failure points and line numbers, demonstrating that structured testing is far superior to random print-statement debugging.

2. ***Architectural Evaluation:*** Knowing what you now know about dynamic array reallocations and shift-based element deletions, what linear architecture alternative (e.g., Linked Lists, Queues, Stacks) would you recommend for systems that require frequent mid-list insertions without reallocating buffers?

   ANSWER: For systems requiring frequent mid-list insertions or deletions without triggering costly buffer reallocations, a Doubly Linked List (`std::list`) is the optimal alternative. Because linked lists manage nodes via pointers rather than contiguous memory blocks, mid-list insertions and deletions execute via simple $O(1)$ pointer relinking operations, avoiding both element shifting and expensive memory reallocations.

3. ***Professional Growth:*** How has evaluating another programmer's code during the Peer Code Review altered your approach to writing clean, well-documented C++ code for your upcoming Milestone 1 project?

   ANSWER: Auditing legacy code highlighted how quickly cryptic variable names, missing scope checks, and unencapsulated globals create technical debt and undermine maintainability. Moving forward into Milestone 1, I will adopt strict RAII principles with class encapsulation, enforce defensive bounds-checking on all inputs, and stick to unified naming conventions to produce clean, audit-ready code